

Task 1 — Web App QA & Debug Report

Application Under Test	RealWorld Conduit Web App
Target URL	https://demo.realworld.show/
Test Date	September 29, 2026
Auditor / QA Specialist	Antigravity QA Engineering Team
Scope	Full-flow end-to-end audit (Sign-up, Authentication, Profile Management, Article CRUD, Comments CRUD, Tagging, Accessibility, Security, and Session Lifecycle)

1. Executive Summary

This report documents the quality assurance, usability, accessibility, and vulnerability assessment conducted on the **RealWorld Conduit** demo application (demo.realworld.show).

A comprehensive test suite was executed covering all critical user journeys using automated browser instrumentation (Playwright Chromium) and manual verification. While core happy paths function, multiple functional defects, validation inconsistencies, accessibility violations, and UX oversights were discovered that prevent the application from being enterprise/production-ready.

2. Test Execution Log (Step-by-Step Testing Lifecycle)

The following sequence of steps was executed during the test audit:

1. Phase 1: Initial Discovery & Landing Page Audit

- Navigated to <https://demo.realworld.show/>.
- Verified page title, hero banner, Global Feed rendering, and Popular Tags sidebar.
- Tested interactive tag filtering (e.g., clicking the `javascript` tag pill) and switching back to Global Feed tab.
- Audited semantic HTML, anchor attributes, and image alt text for accessibility compliance.

2. Phase 2: User Registration & Boundary Validation (Sign Up)

- Navigated to `/register`.
- Tested initial form state (submit button disabled state).
- Tested blank and partial input submission behavior.
- Tested invalid email format input (`invalid_email_no_at`) and password boundary conditions.
- Registered a new unique user (`qa_tester_XXXXX`) with valid credentials.

3. Phase 3: Profile Configuration & Asset Handling (Settings)

- Navigated to `/settings`.
- Updated profile biography and submitted an invalid/unreachable avatar URL (<https://invalid-image-url-does-not-exist-xyz.com/pic.jpg>).
- Inspected profile view at `/profile/<username>` to evaluate image failure handling and UI fallback degradation.

4. Phase 4: Content Authoring & Publishing (New Article / Editor)

- Navigated to `/editor`.
- Tested empty field submissions and client-side validation triggers.
- Filled article with title, description, rich Markdown syntax (headings, code blocks, lists), and embedded inline markup.
- Tested tag input behavior: verified tag addition on Enter, tested duplicate tag inputs (`qa`, `qa`).
- Published article and verified slug routing.

5. Phase 5: Content Interaction & Lifecycle (Comments, Edit, Delete)

- Tested empty comment submission on `/article/<slug>`.
- Posted a verified test comment and verified real-time list update.
- Deleted the comment and verified immediate DOM removal.
- Clicked "Edit Article", modified markdown body, and re-published.
- Executed "Delete Article", confirmed the browser dialog, and verified redirection back to the homepage feed.

6. Phase 6: Session Management & Authentication Recovery (Logout / Login)

- Navigated to `/settings` and clicked "Or click here to logout".
- Navigated to `/login` and tested rejected authentication with invalid credentials.
- Logged in with correct credentials and verified dashboard session restoration.

3. QA Issues & Bug Summary

#	Title	Severity
1	Duplicate Tag Pills Allowed Without Client De-duplication in Article Editor	Medium
2	Broken Image Display Without Fallback on Invalid Avatar URL	Medium
3	Unlabeled / Empty Anchor Links Violating WCAG Accessibility (a11y)	Medium
4	Registration Form Button Enables on Invalid Email Without Real-Time Client Validation	Low / Medium
5	Empty Comment Submission Allowed Without Client-Side Field Validation	High
6	Lack of Password Confirmation Field & Weak Password Policy on Sign-Up	High

Detailed Findings

Bug #1: Duplicate Tag Pills Allowed Without Client De-duplication in Article Editor

Severity	Medium
Steps to Reproduce	1. Navigate to /editor. 2. In the "Enter tags" input field, type qa and press Enter. 3. In the same input, type qa again and press Enter.
Expected	The editor should deduplicate tags (ignore duplicate or highlight existing tag).
Actual	Multiple identical tag chips (qa, qa) are created and appended to the tag list, allowing duplicate tags in payload.
Suspected Cause	The tag insertion handler appends directly to the tags array via <code>tags.push(newTag)</code> or equivalent without checking <code>tags.includes(newTag)</code> or using a Set.

Bug #2: Broken Image Display Without Fallback on Invalid Avatar URL

Severity	Medium
Steps to Reproduce	1. Navigate to /settings. 2. Enter a non-existent / broken image link in "URL of profile picture". 3. Click "Update Settings". 4. Navigate to user profile page /profile/<username> or observe top navbar.
Expected	If the avatar URL fails to load, the UI should gracefully fall back to a default avatar placeholder icon.
Actual	Browser renders a broken image placeholder / empty box, breaking the UI layout.
Suspected Cause	Missing <code>onError</code> / <code>onerror</code> event listener on <code></code> components to reset <code>src</code> to a fallback placeholder image.

Bug #3: Unlabeled / Empty Anchor Links Violating WCAG Accessibility (a11y)

Severity	Medium
Steps to Reproduce	1. Inspect the homepage and article list DOM (https://demo.realworld.show/). 2. Inspect anchor tags wrapping profile pictures and branding.
Expected	All interactive anchor tags must have descriptive text, <code>aria-label</code> , or an accessible name for screen readers.
Actual	Multiple anchor tags contain empty inner text without <code>aria-label</code> or <code>title</code> attributes.
Suspected Cause	Developer wrapped images or icons inside anchor tags without providing text content or <code>aria-label</code> / <code>aria-hidden</code> attributes.

Bug #4: Registration Form Button Enables on Invalid Email Without Real-Time Client Validation	
Severity	Low / Medium
Steps to Reproduce	1. Navigate to /register. 2. Fill Username and Password. 3. Enter an invalid email without @ (e.g. invalid-email). 4. Observe the submit button state.
Expected	Submit button should remain disabled or provide inline format error feedback before submission.
Actual	Submit button becomes enabled regardless of invalid format, submitting an invalid payload and requiring unnecessary server roundtrips.
Suspected Cause	Form validity check only evaluates truthy non-empty string presence rather than an RFC 5322 regex / standard HTML5 email pattern.

Bug #5: Empty Comment Submission Allowed Without Client-Side Field Validation	
Severity	High
Steps to Reproduce	1. Navigate to any published article page (/article/<slug>). 2. Focus the comment input without typing any text. 3. Click the "Post Comment" button.
Expected	"Post Comment" button should be disabled when the textarea is empty/whitespace, or display an inline validation warning.
Actual	"Post Comment" is enabled and clickable, dispatching an empty payload to the API and returning an unhandled 422 error.
Suspected Cause	The comment form lacks a client-side validation guard (<code>comment.trim().length > 0</code>) on button state and form submit handler.

Bug #6: Lack of Password Confirmation Field & Weak Password Policy on Sign-Up	
Severity	High
Steps to Reproduce	1. Navigate to /register. 2. Notice form fields: only Username, Email, and a single Password field are present. 3. Enter a 1-character password (e.g., 1).
Expected	Registration should require password confirmation to avoid accidental typos and enforce baseline security complexity.
Actual	A single entry field allows users to lock themselves out with typos; weak passwords are allowed without client strength indicators.
Suspected Cause	Non-developer / vibe-coded scaffold omitted the password confirmation field and client-side password complexity regex.

4. Root Cause Analysis (Deep Dive on Issue #1)

Selected Issue: Duplicate Tag Pills Allowed in Article Editor

What is Happening & Why

In the Article Editor component (/editor), when a user enters a tag string into the "Enter tags" input field and presses Enter, the client appends the value directly into the component's internal `tagList` state array. The input event handler fails to sanitize or check whether the tag already exists in the collection prior to appending. Because no set deduplication or array inclusion guard (`tagList.includes(trimmedTag)`) is applied, the UI renders duplicate `<span class="tag-default tag-pill">` elements with redundant removal buttons. Furthermore, when the article is submitted, this redundant array is serialized into the JSON payload sent to the backend POST

/api/articles endpoint, causing database bloat, indexing redundancies, and UI display anomalies.

How to Fix It (Recommended Solution)

The tag addition handler in the frontend editor component should trim whitespace, ignore empty strings, and verify uniqueness before mutating the state. Specifically, checking `!tagList.includes(newTag)` before appending guarantees that tags remain unique and clean before they are sent to the backend API.

5. Visual Testing Evidence (Screenshots)

The screenshot image files referenced by the original report were not included with the upload, so each figure below is shown as a placeholder with its original filename.

Figure 1: Bug #1 Reproduction — Article Editor with Duplicate Tags (qa, qa)

[Screenshot placeholder]
screenshots/10_editor_filled_tags.png

Figure 2: Bug #4 & #6 — Registration Form with Single Password & Email Validation State

[Screenshot placeholder]
screenshots/04_signup_empty_form.png

Figure 3: Article Published View & Content Rendering

[Screenshot placeholder]
screenshots/11_article_view.png

Figure 4: Comment Interaction & Lifecycle

[Screenshot placeholder]
screenshots/13_comment_posted.png

Figure 5: Landing Page & Tag Navigation

[Screenshot placeholder]
screenshots/01_landing_page.png

Figure 6: Logged Out State

[Screenshot placeholder]
screenshots/18_logged_out.png

6. Recommendations for Stakeholders

1. **Implement Client-Side Form Validation:** Add standard HTML5 / regex constraints and immediate visual feedback to registration, login, and editor forms before network requests are dispatched.
2. **Standardize Asset Fallbacks:** Enforce default SVG/PNG avatar fallbacks across the entire application using component-level error handlers.
3. **Enhance Web Accessibility (WCAG 2.1 AA):** Add `aria-label` attributes to icon-only links, buttons, and inputs to ensure screen reader compatibility.
4. **Security & Password Policies:** Introduce a "Confirm Password" field and enforce minimum complexity rules (min 8 characters, special character, uppercase/lowercase).